

Variables and Data Types

Features for declaring and managing variables and data types in Java.

Features: byte, short, int, long, float, double, char, boolean, String, final, null

Feature	Uses	Example	Practical Uses
byte	8-bit integer (-128 to 127).	<code>byte b = 100;</code>	Storing small numbers like age or flags.
short	16-bit integer (-32,768 to 32,767).	<code>short s = 1000;</code>	Handling moderate-sized numbers like counters.
int	32-bit integer.	<code>int i = 50000;</code>	General-purpose integer for calculations.
long	64-bit integer.	<code>long l = 1000000L;</code>	Large numbers like timestamps or IDs.
float	32-bit floating-point.	<code>float f = 3.14f;</code>	Approximate decimals for graphics.
double	64-bit floating-point.	<code>double d = 3.14159;</code>	Precise decimals for scientific calculations.
char	16-bit Unicode character.	<code>char c = 'A';</code>	Storing single characters like initials.
boolean	True/false values.	<code>boolean isActive = true;</code>	Conditional logic like user status.
String	Immutable sequence of characters.	<code>String str = "Hello";</code>	Text handling in user input or output.
final	Declares a constant variable.	<code>final int MAX = 100;</code>	Defining immutable constants like limits.
null	Represents no value.	<code>String str = null;</code>	Initializing objects or clearing references.

Operators

Features for performing operations on variables.

Features: arithmetic (+, -, *, /, %), assignment (=), comparison (==, !=, <, >, <=, >=), logical (&&, ||, !), increment/decrement (++ , --), ternary (? 🤖)

Feature	Uses	Example	Practical Uses
+ (arithmetic)	Adds numbers or concatenates strings.	<code>int sum = 5 + 3;</code>	Calculating totals in apps.
- (arithmetic)	Subtracts numbers.	<code>int diff = 5 - 3;</code>	Computing differences like discounts.
* (arithmetic)	Multiplies numbers.	<code>int prod = 5 * 3;</code>	Scaling values in algorithms.
/ (arithmetic)	Divides numbers.	<code>double div = 10.0 / 3;</code>	Calculating averages or ratios.
% (arithmetic)	Returns remainder of division.	<code>int mod = 10 % 3;</code>	Checking even/odd or cycling indices.
= (assignment)	Assigns value to variable.	<code>int x = 5;</code>	Storing user input or results.
== (comparison)	Checks equality.	<code>if (x == y)</code>	Validating form inputs.

Feature	Uses	Example	Practical Uses
<code>!=</code> (comparison)	Checks inequality.	<code>if (x != y)</code>	Ensuring distinct values.
<code><, >, <=, >=</code>	Compares values.	<code>if (x < 10)</code>	Filtering data like age checks.
<code>&&</code> (logical)	Logical AND.	<code>if (x > 0 && y > 0)</code>	Combining conditions in validation.
		(logical)	Logical OR.
<code>!</code> (logical)	Logical NOT.	<code>if (!isValid)</code>	Inverting conditions.
<code>++</code>	Increments by 1.	<code>x++;</code>	Loop counters or step increments.
<code>--</code>	Decrements by 1.	<code>x--;</code>	Decreasing counters in timers.
<code>?:</code> (ternary)	Conditional expression.	<code>String result = x > 0 ? "Positive" : "Negative";</code>	Concise conditionals in UI.

Control Flow

Features for controlling program execution.

Features: if, else, switch, for, while, do-while, break, continue, return

Feature	Uses	Example	Practical Uses
if	Executes block if condition is true.	<code>if (x > 0) { System.out.println("Positive"); }</code>	Validating user input.
else	Executes block if condition is false.	<code>if (x > 0) {} else { System.out.println("Non-positive"); }</code>	Handling alternative cases.
switch	Multi-way branching based on value.	<code>switch (day) { case 1: System.out.println("Monday"); break; }</code>	Menu options or state handling.
for	Loops with counter.	<code>for (int i = 0; i < 5; i++) { System.out.println(i); }</code>	Iterating over arrays or lists.
while	Loops while condition is true.	<code>while (x > 0) { x--; }</code>	Processing until condition changes.
do-while	Loops at least once.	<code>do { x--; } while (x > 0);</code>	Ensuring one iteration in menus.
break	Exits loop or switch.	<code>for (int i = 0; i < 5; i++) { if (i == 3) break; }</code>	Early loop termination.
continue	Skips current loop iteration.	<code>for (int i = 0; i < 5; i++) { if (i == 2) continue; }</code>	Skipping invalid data in loops.
return	Exits method with value.	<code>return x * 2;</code>	Returning results from calculations.

Classes and Objects

Features for object-oriented programming in Java.

Features: class, new, this, constructor, method, instance variable, static, extends, super

Feature	Uses	Example	Practical Uses
class	Defines a blueprint for objects.	<code>class Person { String name; }</code>	Modeling entities like users or products.
new	Creates a new object instance.	<code>Person p = new Person();</code>	Instantiating objects like database connections.
this	Refers to current object.	<code>this.name = name;</code>	Disambiguating instance variables in constructors.
constructor	Initializes new objects.	<code>Person(String name) { this.name = name; }</code>	Setting initial object state.
method	Defines behavior for a class.	<code>void greet() { System.out.println("Hi"); }</code>	Encapsulating logic like calculations.
instance variable	Stores object-specific data.	<code>class Person { String name; }</code>	Storing attributes like user details.
static	Defines class-level members.	<code>static int count = 0;</code>	Tracking shared data like object counts.
extends	Inherits from a parent class.	<code>class Student extends Person { }</code>	Reusing code in hierarchies like vehicles.
super	Accesses parent class members.	<code>super(name);</code>	Calling parent constructors or methods.

Access Modifiers

Features for controlling access to class members.

Features: public, private, protected, default (package-private)

Feature	Uses	Example	Practical Uses
public	Accessible from everywhere.	<code>public int x;</code>	Exposing methods like APIs.
private	Accessible only within class.	<code>private String name;</code>	Encapsulating sensitive data like passwords.
protected	Accessible in same package and subclasses.	<code>protected int id;</code>	Sharing data with subclasses.
default	Accessible within same package if no modifier.	<code>int count;</code>	Package-level utilities.

Exception Handling

Features for managing errors.

Features: try, catch, finally, throw, throws

Feature	Uses	Example	Practical Uses
try	Defines block to monitor for exceptions.	<code>try { int x = 1 / 0; }</code>	Wrapping risky operations like file I/O.
catch	Handles specific exceptions.	<code>catch (ArithmeticException e) { System.out.println("Error"); }</code>	Handling division by zero or null pointers.
finally	Executes code regardless of exception.	<code>finally { System.out.println("Done"); }</code>	Closing resources like database connections.
throw	Throws an exception manually.	<code>throw new IllegalArgumentException("Invalid");</code>	Custom error handling in methods.
throws	Declares exceptions a method may throw.	<code>void readFile() throws IOException</code>	Specifying file operation risks.

Standard Library Basics

Commonly used classes and methods from Java’s standard library.

Features: ArrayList, StringBuilder, Math, System.out.println, Scanner

Feature	Uses	Example	Practical Uses
ArrayList	Dynamic array for storing elements.	<code>ArrayList<Integer> list = new ArrayList<>(); list.add(1);</code>	Managing dynamic lists like user inputs.
StringBuilder	Mutable string for efficient concatenation.	<code>StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");</code>	Building strings in loops.
Math	Provides mathematical functions.	<code>double pi = Math.PI; int max = Math.max(5, 10);</code>	Calculations like rounding or trigonometry.
System.out.println	Prints to console.	<code>System.out.println("Hello");</code>	Debugging or logging output.
Scanner	Reads input from sources like console.	<code>Scanner sc = new Scanner(System.in); int n = sc.nextInt();</code>	Reading user input in CLI apps.

Loops and Iteration

Features for iterating over data.

Features: for, enhanced for (for-each), while, do-while (already covered in Control Flow)

Feature	Uses	Example	Practical Uses
for	Loops with counter.	<code>for (int i = 0; i < 5; i++) { System.out.println(i); }</code>	Iterating arrays or ranges.
enhanced for	Iterates over arrays/collections.	<code>for (int x : array) { System.out.println(x); }</code>	Processing lists like student records.

Methods and Overloading

Features for defining and reusing methods.

Features: method declaration, return type, void, method overloading

Feature	Uses	Example	Practical Uses
method declaration	Defines reusable code block.	<code>int add(int a, int b) { return a + b; }</code>	Encapsulating logic like calculations.
return type	Specifies method's output type.	<code>double getArea() { return 3.14 * r * r; }</code>	Returning computed values.
void	Indicates no return value.	<code>void print() { System.out.println("Hi"); }</code>	Performing actions like logging.
method overloading	Multiple methods with same name, different parameters.	<code>int add(int a, int b); double add(double a, double b);</code>	Flexible APIs for different input types.